

Neural means and kernel corrections for operator learning

Yitzchak Shmalo*

July 2026

Abstract

We study how neural networks and exact kernel methods combine in operator learning: an accurate neural mean, trained in the metric the application reports, with an exact Matérn solve applied to its residual or to its learned features. Two public benchmarks with published baselines sit in opposite regimes and the same components explain both. On the structural-mechanics problem of de Hoop et al. the pipeline reaches 4.55% relative test error, matching the best published result, and the reason no method moves much further is measurable: the residuals of every architecture we train correlate above 0.86, and their shared component concentrates where the finite element data are least accurate and responds neither to capacity, diversity, nor sample size. On the OCO-2 radiative-transfer emulation problem of Lamminpää et al. the same kernel that trails the network tenfold on the raw input overtakes it on the network’s learned features, and the resulting surrogate beats the published kernel-flow emulator on its own test points on both of that problem’s metrics (3.8% against 16.9% on reduced radiances, 0.028% against 0.045% on the reconstructed spectrum). Each stage of the pipeline carries a supporting result, from a second-moment identity that predicts the stacking outcome from measured residual correlations, through an optimal-recovery bound and rates separating the metric from the representational part of a network’s advantage over a fixed kernel, to a distribution-free coverage guarantee for the reported uncertainty, which is the only uncertainty signal that survives our tests.

1 Introduction

Operator learning constructs fast surrogates for the solution maps of parametric partial differential equations and physical forward models from data. Two families dominate the field: neural operators, which learn their own representations, and kernel or Gaussian-process methods, which come with exact solves and error certificates. This paper studies how the two combine, on two public problems with published baselines that turn out to sit at opposite ends of the neural-versus-kernel spectrum: the structural-mechanics benchmark of de Hoop et al. [5], where the families tie and the question is what their coupling adds, and the OCO-2 radiative-transfer emulation problem of Lamminpää et al. [9], where a fixed kernel trails a network by an order of magnitude until it is handed the network’s learned features. The same small set of components, and the same supporting theory, account for both.

Among the standard test problems in the operator-learning literature, the structural mechanics benchmark introduced by de Hoop et al. [5] has proved unusually resistant to improvement. The task is to learn the map from a boundary load applied to the top edge of an elastic plate to the interior von Mises stress field computed by a finite element solver; inputs are sampled from a

*Department of Mathematics, Pennsylvania State University. yms5281@psu.edu

Gaussian field, and errors are reported as mean relative L^2 on a held-out set. With 20000 training samples, the four architectures compared by de Hoop et al. [5] land within half a percentage point of each other (PARA-Net 4.55%, PCA-Net 4.67%, FNO 4.76%, DeepONet 5.20%), and the optimal-recovery kernel method of Batlle et al. [2], which matches or beats neural operators on most of the other benchmarks in that study, does not do so here (5.18%); the authors single out this problem as the most challenging one for their method. More recently Mora et al. [14] revisited the problem in a low-data regime and reported 6.49% with a Gaussian process whose mean is an FNO, against 6.62% for the FNO alone, with 1250 training samples. The picture suggested by these numbers is a plateau around 4.5% that neither pure neural nor pure kernel methods penetrate.

This paper does two things with that plateau. First it reaches the front of it: a residual coupling of neural and kernel methods attains 4.55% relative test error under the 20000-sample protocol, matching the best published result (PARA-Net, 4.55%) to within run-to-run noise and below every other published method, and 5.38% under the 1250-sample protocol against a published best of 6.49%, a clear margin there. Second, and we think more usefully, it explains why the front of the band is where it is and why further movement should not be expected from modeling. The residuals of maximally different surrogates on this benchmark are nearly the same residual: across three architecture families and two training losses, per-sample residual correlations lie between 0.86 and 0.96, and the component shared by all members carries over ninety percent of the residual energy, concentrates at the corners of the loaded edge where the finite element data are least reliable, and is statistically independent of every input statistic we compute. The error of a fixed recipe is also nearly flat in the training set size. Taken together (Section 5.3), the evidence reads the published plateau as a property of the benchmark’s data rather than of the surrogate class, and our final number as sitting within a few hundredths of what this dataset can certify. The paper also returns a pointwise uncertainty estimate, whose reach and limits we characterize carefully rather than oversell. The pipeline combines three ingredients, none of which is exotic on its own.

First, representation. The benchmark ships its input as a 41×41 array in which the one-dimensional load is copied across 41 columns. Treating the input as what it is, a function of one variable sampled at 41 points, changes what is easy: kernel methods on $\mathbb{R}^{41}$ with 19000 training points are exact linear-algebra computations (a single Cholesky factorization of the Gram matrix), and the guesswork of choosing a dimension reduction for the input disappears. Some of the published baselines flatten the redundant 41×41 input or reduce it by PCA; none of the numbers above changes materially under such reformattings, so we do not attribute the gap we open to this step alone, but it is what makes the rest of the pipeline cheap and transparent.

Second, an accurate neural mean. The neural-mean slot is architecture-agnostic; we instantiate it with a residual multilayer perceptron and, as a second member, the same network fed the kernel method’s own prediction as an extra channel, so that it learns to refine the kernel rather than to reproduce it. Both are trained with a loss equal to the reported metric, with reflection augmentation and reflected test-time averaging. On this benchmark the refiner is among the strongest single models we obtain, and a convex combination of the two with the kernel predictor is near the top of the published band. A Fourier neural operator, a UNet, and an MSE-trained variant of the MLP join the ensemble in Section 5.3, where their role turns out to be diagnostic as much as additive. The reflection steps are backed by an elementary but useful fact (Proposition 4.1): if the target operator is equivariant under an isometric involution and the input law is invariant, symmetrizing any estimator cannot increase a convex risk. We verify the

equivariance directly on the data by locating near-mirror input pairs and checking their outputs (Section 2).

Third, a kernel correction of the residual. The stacked ensemble is treated as the mean of a Gaussian process on the 41-dimensional load space, and the residual operator is regressed by kernel ridge with a Matérn kernel, scale and nugget tuned by validation; a second, smaller correction uses the trained networks’ penultimate features as kernel inputs. This stage is where the theory earns its keep. The corrected surrogate satisfies a pathwise bound (Theorem 4.6) of the form $\|\text{error at } u\| \leq \|G - m\|_K \cdot P_\lambda(u)$, the product of the RKHS norm of the residual operator and the Gaussian process posterior standard deviation. The operator factor explains why one should regress residuals rather than stresses: the computable proxy for it, the RKHS norm of the fitted interpolant, drops by about $271\times$ when the kernel is moved from raw targets to ensemble residuals. The kernel is asked to represent only what the networks left behind, and what they leave behind is much smaller as the kernel measures it, so the same design gives a much tighter certificate.

The design factor P_λ is the natural candidate for a reported uncertainty, and here our finding is a cautionary one. As a pointwise ranking of the corrected surrogate’s error, P_λ is weak: the neural mean supplies most of the error, and its error pattern is not a function of the input-space geometry that P_λ measures. Worse, the benchmark’s relative-error metric is confounded by output amplitude, so the raw correlation of P_λ with relative error is even slightly negative. The disagreement of our ensemble is no more informative, its members being architecturally alike and so wrong in the same places. What survives is what should: the conformal rescaling of P_λ on held-out data is distribution-free and attains its nominal coverage on test. We report this in full in Section 5.4 because the gap between a valid bound and a useful indicator is exactly the kind of thing benchmark papers tend to gloss.

We also examine two further points of contact between the pipeline and theory. The kernel-flow regularizer of Owhadi and Yoo [16], Yoo and Owhadi [22], which we include as a variant in the training of the neural means, admits an exact description: the ℓ^2 kernel-flow batch loss is an unbiased estimator, over the random half-split, of the leave-half-out prediction error of a kernel method built on the network’s features (Proposition 4.2), so adding it trains the features for the role they later play in the correction stage. And the nugget selected by cross-validation admits a spectral reading: an exact identity (Lemma 4.8) expresses the effective dimension of the kernel solve through the Stieltjes transform of the Gram spectrum, which for our learned features exhibits the bulk-plus-spikes shape familiar from random matrix theory; the cross-validated nugget sits where the effective dimension has absorbed the spikes and leading bulk. Neither observation is deep, but both replace folklore with checkable statements, and both checks pass on this problem.

The paper is organized as follows. Section 2 describes the benchmark, the published results we compare against, and the data-driven verification of the mirror symmetry. Section 3 specifies the pipeline. Section 4 states the supporting results, one per stage of the pipeline, with proofs in Appendix A. Section 5 reports the main comparisons, ablations that attribute the improvement to its sources, the diversity experiment and the anatomy of the shared residual, the uncertainty calibration study, and the spectral analysis. Section 7 discusses limitations and the relation to Gaussian process emulation practice at scale, including the line of work on kernel-flow-based emulators for remote sensing retrievals [9].

2 Benchmark, protocol, and related work

2.1 Problem and data

The dataset originates in the cost-accuracy study of de Hoop et al. [5] and is the one distributed with Batlle et al. [2] and Mora et al. [14]. An isotropic elastic plate occupies $D = (0, 1)^2$; the displacement field w satisfies $\nabla \cdot \sigma = 0$ with a fixed constitutive law, displacement conditions on the bottom and lateral parts of the boundary, and a prescribed normal traction $\bar{t}$ on the top edge $\Gamma_t = [0, 1] \times \{1\}$. The quantity of interest is the von Mises stress field σ_v on D . The learning task is the map $G : \bar{t} \mapsto \sigma_v$. Loads are drawn from the Gaussian field $\mathcal{GP}(100, 400^2(-\Delta + 3^2 I)^{-1})$ with homogeneous Neumann boundary conditions for the Laplacian; outputs are finite element solutions interpolated to a regular 41×41 grid, and the load is sampled at 41 points. The distributed file contains 40000 input/output pairs; the input array stores the load broadcast along the second grid coordinate, which we verified is an exact copy (maximal deviation 0 across all 40000 samples), so all our methods consume the load as a vector in $\mathbb{R}^{41}$.

Errors are mean relative L^2 errors over the test set,

$$\frac{1}{N_{\text{test}}} \sum_n \frac{\|\hat{G}(u_n) - G(u_n)\|_2}{\|G(u_n)\|_2},$$

computed on grid values in double precision. Quadrature weighting (trapezoidal instead of plain vector norms) changes our reported numbers by less than 0.02 percentage points, and we report the plain-norm convention of the prior work.

2.2 Protocols and published results

Two regimes appear in the literature, and we follow both. In the *high-data* protocol the first 20000 samples are available for training and the last 20000 form the test set; de Hoop et al. [5] report, at 20000 training samples, 4.55% (PARA-Net), 4.67% (PCA-Net), 4.76% (FNO) and 5.20% (DeepONet), and Batlle et al. [2] report 5.18% for their optimal-recovery kernel method (Matérn/rational quadratic; 27.11% for the linear kernel) on the same task. Within the 20000-sample budget we hold out the last 1000 samples (of a fixed permutation) for model selection and stacking, and train on the remaining 19000, so no method of ours sees more than the 20000 training samples used by the baselines. In the *low-data* protocol of Mora et al. [14], 1250 samples are available and the same 20000-sample test set is used; their table gives 8.70% (DeepONet), 6.62% (FNO), 6.95% (the kernel method of Batlle et al. 2), 6.74% (their zero-mean GP), 7.12% (DeepONet-mean GP) and 6.49% (FNO-mean GP). In this regime we carve the validation split (250 samples) out of the 1250, so training uses 1000 samples and every choice made by the pipeline is informed by the 1250 available labels only.

2.3 A data-driven check of the mirror symmetry

The continuous problem is invariant under $x_1 \mapsto 1 - x_1$: the domain and boundary partition are symmetric, and the input law has a symmetric covariance. On the grid, reflecting the load (S) should reflect the stress field along the first grid coordinate (T), i.e. $G(Su) = TG(u)$. Rather than assume this, we test it on the data. For each of the first 200 samples we searched all 40000 loads for the nearest neighbor of the reflected load Su_i and compared the corresponding outputs. For the five best-matching pairs, the input mismatch $\|Su_i - u_j\|/\|u_j\|$ ranges over 0.27–0.32,

while the output mismatch $\|Tv_i - v_j\|/\|v_j\|$ ranges over 0.085–0.14; reflecting along the second coordinate instead gives output mismatches of order one. Outputs of near-mirror inputs are far closer than the inputs themselves, which is what equivariance plus a Lipschitz solution map predicts, and the effect singles out the correct output reflection axis. A complementary check on the input law: the empirical mean and covariance of the training loads are invariant under reflection to within 1.1% and 1.5% respectively, as required for Proposition 4.1. Consistently with all of this, reflection averaging at test time improves every model we trained (Section 5), as Proposition 4.1 says it must on average when the symmetry holds.

2.4 Related work

The benchmark sits at the meeting point of three lines of work. *Neural operators* learn maps between function spaces: DeepONet [12] with its branch/trunk factorization, the Fourier neural operator [11] and the broader neural-operator framework [8], and the reduced-basis PCA-Net and PARA-Net architectures assembled for the cost–accuracy study of de Hoop et al. [5]. These methods are fast and mesh-flexible but do not by themselves come with error control, and it is their numbers on this problem that define the plateau we start from. *Kernel and Gaussian process methods* for operator learning approach the same maps through reproducing kernels: the optimal-recovery framework of Batlle et al. [2], with its convergence theory and a-priori bounds, is the most direct comparison, and it is competitive with neural operators on most of the benchmarks it considers. Random-feature variants [4] and the vector-valued kernel formulations reviewed there extend the reach of this family. *Hybrids* that place a neural network and a kernel in the same estimator are the closest relatives of our pipeline: Mora et al. [14] use a neural operator as the mean of a Gaussian process and fit the two together, and this is the work we most directly build on, differing in that we fit the mean first and the kernel after, tune by cross-validation rather than marginal likelihood, and solve the correction exactly at nineteen thousand points.

Two further threads inform the design. The kernel-flow method [16] learns a kernel from data by minimizing a half-sample cross-validation loss, and its use as a regularizer for the inner layers of a network [22] is exactly the term we analyze in Proposition 4.2; kernel flows have since been used at scale as emulators for physical forward models, for instance in atmospheric radiative transfer retrievals [9] and in the inference of convective-storm structure from satellite observations [17], where presenting the input in its physical parametrization and adapting the kernel to data are what make the emulator accurate. Our pipeline follows the same instinct, with the smoothing of the target performed by a neural ensemble rather than by a hand-chosen reduction. Finally, the effective-dimension reading of the kernel solve (Lemma 4.8) draws on the random-matrix description of kernel Gram spectra [7] and on the classical Marčenko–Pastur [13] and spiked-covariance [1] laws; the same effective dimension governs the generalization of kernel ridge regression [3].

3 Method

The pipeline has three stages: neural means, stacking, and kernel corrections. All components operate on the load vector $u \in \mathbb{R}^{41}$ (standardized by training statistics) and produce stress fields in $\mathbb{R}^{41 \times 41}$; all networks are trained with the reported metric as the loss, i.e. the per-sample relative L^2 error in original units, which we found mildly but consistently better than mean squared error on normalized targets.

3.1 Neural means

Residual MLP. The primary mean is a residual multilayer perceptron: three or five residual SiLU blocks of width 1024–1536 mapping $\mathbb{R}^{41} \rightarrow \mathbb{R}^{1681}$ directly. This is essentially PARA-Net [5] with a modern training recipe, and it also supplies the penultimate features used by the feature-space kernel stage.

Kernel-conditioned refiner. The second mean is the same residual network given an extra input channel: the kernel method’s predicted stress field for the same load, concatenated with the load. It therefore learns to correct the kernel predictor rather than to reproduce the map from scratch, and it is the strongest single model in our study. During training the refiner reads *out-of-fold* kernel predictions (four-fold, so the kernel channel is never fit on the target sample), and at test time the full-data kernel prediction; this keeps the channel honest.

Other instances. The mean slot is not tied to a particular architecture. We also use a Fourier neural operator [11] that consumes the broadcast load as a field, a UNet on the same field representation, and an MSE-trained variant of the MLP; all are drop-in means, all enter the ensemble of Section 5.3, and their pairwise error correlations are one of the measurements of this paper. We additionally implement an encoder–decoder transformer [6, 18] that tokenizes the 41 load samples and decodes the field by cross-attention at the grid points; it is a natural fourth architecture family, but we were unable to train it to the level of the others on the hardware available for this study and report the ensemble without it (Section 5.3 returns to this).

All means are trained with AdamW and a cosine schedule, batches of 128–256, for up to a few hundred epochs (Appendix B lists every hyperparameter). During training each batch is reflected with probability 1/2 (load reversed, target field reflected along the first grid coordinate); at test time each model is evaluated as $\frac{1}{2}(f(u) + Tf(Su))$. Proposition 4.1 guarantees the test-time step cannot hurt on average, and the ablation confirms small consistent gains from both steps. Optionally, the training loss carries a kernel-flow term βe_2 computed from the batch (Section 4.2) with a Gaussian kernel on pooled penultimate features whose log-bandwidth is trained jointly; this variant appears in the ablation.

3.2 Stacking

With M trained models $f_1, \dots, f_M$ we form the convex combination $m(u) = \sum_m w_m f_m(u)$, $w \in \Delta^{M-1}$, minimizing the validation relative error; the weights are initialized at the simplex minimizer of the measured second-moment matrix of Proposition 4.4 and polished by a short search on the 1000 held-out samples (250 in the low-data protocol). A per-pixel variant allows the weights (plus an intercept) to vary over the grid, ridge-fitted on half the validation split and accepted only when it beats the global weights on the other half; it is the variant the final pipeline uses. Stacking on a held-out split rather than uniform averaging costs nothing, guards against a weak member [21], and is statistically almost free at this size (Proposition 4.3).

3.3 Kernel corrections

The stacked mean is then corrected by kernel ridge regression of its residuals. Let $X = (u_1, \dots, u_n)$ be the training loads (standardized coordinatewise), $R \in \mathbb{R}^{n \times 1681}$ the matrix of training residuals $v_i - m(u_i)$, and k a Matérn-5/2 kernel $k(u, u') = \kappa(\|u - u'\|/s)$. The correction is

$$\hat{r}(u) = k(u, X) \alpha, \quad \alpha = (K + n\lambda I)^{-1} R,$$

with (s, λ) chosen on the validation split from a small grid (s as a multiple of the median pairwise distance, $\lambda \in \{10^{-7}, 10^{-5}, 10^{-3}\}$; the grid is searched on an 8000-sample subsample and the chosen pair is refit on all of X). The corrected surrogate is $m + \hat{r}$. A second corrective stage repeats the construction with the kernel evaluated on deep features, the concatenated penultimate activations of the ensemble members (reflection-averaged, standardized), fitted to the residuals of $m + \hat{r}$; it contributes a smaller improvement and is included when it helps on validation. At $n = 19000$ the exact solve is a single Cholesky factorization of a 19000×19000 matrix, about a minute in double precision on a laptop CPU, and prediction is two matrix products; no inducing points, preconditioners, or stochastic solvers are involved. The Gaussian process reading of the same formulas supplies the posterior standard deviation $P_\lambda(u)$ of (2) at negligible extra cost (one triangular solve per evaluation batch), which is the uncertainty estimate studied in Section 5 and certified by Theorem 4.6.

Two remarks on scope. Nothing in the construction uses properties of this particular PDE beyond the verified reflection symmetry, so the recipe (native input parametrization, symmetrized accurate means, validated kernel correction of the residual, posterior standard deviation as certificate) transfers to other operator learning problems with low-dimensional input parametrizations. And the pipeline degrades gracefully: dropping the feature stage, the stacking, or the corrections recovers progressively simpler methods whose individual numbers appear in the ablation table.

4 Theory

Throughout this section $\mathcal{U} = \mathbb{R}^p$ denotes the (discretized) input space and $\mathcal{V} = \mathbb{R}^q$ the output space, $G : \mathcal{U} \rightarrow \mathcal{V}$ the target operator, μ the input distribution, and $(u_1, v_1), \dots, (u_N, v_N)$ i.i.d. draws with $v_n = G(u_n)$; the data carry no sampling noise in the usual sense, coming from a deterministic solver, though Section 5.3 has something to say about solver error. We write $\ell(w, v) = \|w - v\|_2 / \|v\|_2$ for the relative error and $R(\hat{G}) = \mathbb{E}_{u \sim \mu} \ell(\hat{G}(u), G(u))$ for the risk, which is the quantity reported in Section 5. The results below are largely elementary, but each one licenses a specific design decision in the pipeline of Section 3, and each has a measurable footprint in the experiments: one statement per stage, from the symmetry handling and the kernel-flow term through stacking, the residual correction, its coverage guarantee, and the choice of nugget. Proofs are collected in Appendix A.

4.1 Symmetry averaging does not increase the risk

The elasticity problem behind the benchmark is invariant under the reflection $x_1 \mapsto 1 - x_1$: the domain, the boundary partition, and the constitutive model are all symmetric, the input distribution has a reflection-invariant covariance, and reflecting the load reflects the stress field. On the discrete grid this is expressed by two permutation matrices: $S \in \mathbb{R}^{p \times p}$ reverses the load samples and $T \in \mathbb{R}^{q \times q}$ reverses the rows of the stress field. Both are orthogonal involutions. Section 2 describes a direct data-driven check of the equivariance $G(Su) = TG(u)$; we treat it as an assumption here.

Proposition 4.1 (symmetrization). *Assume $G(Su) = TG(u)$ for μ -a.e. u , that μ is invariant under S , and that T is orthogonal. For a measurable $\hat{G} : \mathcal{U} \rightarrow \mathcal{V}$ define its symmetrization*

$$\hat{G}_S(u) = \frac{1}{2} \left(\hat{G}(u) + T \hat{G}(Su) \right).$$

Then, for every loss $\ell(\cdot, v)$ that is convex in its first argument and satisfies $\ell(Tw, Tv) = \ell(w, v)$,

$$\mathbb{E}_{u \sim \mu} \ell(\widehat{G}_S(u), G(u)) \leq \mathbb{E}_{u \sim \mu} \ell(\widehat{G}(u), G(u)).$$

The relative error satisfies both hypotheses (T orthogonal gives $\ell(Tw, Tv) = \ell(w, v)$). Proposition 4.1 justifies the two uses of the symmetry in the pipeline: averaging each trained model with its reflected evaluation at test time can only help on average, and training on reflection-augmented data is ordinary empirical risk minimization for the symmetrized class. The measured effect is small but strictly positive for every model we trained (Table 3).

4.2 What the kernel-flow regularizer estimates

The kernel-flow (KF) loss of Owhadi and Yoo [16], in the ℓ^2 variant used by Yoo and Owhadi [22] to regularize deep networks, enters our ablation study as an optional term in the training of the neural means. Its motivation is often stated informally (“a kernel is good if half the data predicts the rest”). The following observation makes the statement exact. Let $z_i = \phi_\theta(u_i)$ be the features of a batch $B = \{1, \dots, b\}$ (b even), let k be a positive definite kernel on feature space, and for $C \subset B$, $|C| = b/2$, let I_C denote the k -interpolant of $\{(z_j, v_j) : j \in C\}$. The batch KF loss is

$$e_2(\theta; C) = \sum_{i \in B} \|v_i - I_C(z_i)\|_2^2. \quad (1)$$

Proposition 4.2 (KF loss is a leave-half-out estimate). *Let C be drawn uniformly among the subsets of B of size $b/2$ and assume k restricted to $\{z_i\}_{i \in B}$ is strictly positive definite. Then $e_2(\theta; C) = \sum_{i \in B \setminus C} \|v_i - I_C(z_i)\|_2^2$, and*

$$\mathbb{E}_C [e_2(\theta; C)] = \frac{b}{2} \mathbb{E}_{C, i \sim \text{Unif}(B \setminus C)} [\|v_i - I_C(z_i)\|_2^2],$$

i.e. $\frac{2}{b} e_2$ is an unbiased estimator, over the split randomness, of the expected squared error committed on a held-out point by the kernel predictor trained on a random half of the batch. When (B, C) are resampled at every step, the stochastic gradient $\nabla_\theta e_2(\theta; C)$ is unbiased for $\nabla_\theta \mathbb{E}_{B, C} [e_2(\theta; C)]$ whenever the expectation and derivative commute (e.g. under local Lipschitz domination).

The training loss we minimize for a neural mean is a sum of an empirical risk term and, optionally, βe_2 : the first term is a linear functional of the empirical measure of the batch and measures fit, while Proposition 4.2 shows the second measures the *generalization of a kernel method built on the learned features*. This is the sense in which a KF-regularized network is trained to be a good feature map for the kernel correction that follows.

4.3 Stacking on a held-out split is safe

Between the means and the correction sits one more fitted object: the convex weights of the stacked ensemble, chosen to minimize the empirical risk on the validation split. Two elementary facts justify this step. First, by convexity, a convex combination of predictors is never worse than the corresponding weighted average of their risks, so stacking cannot be hurt by a weak member beyond its weight. Second, the weights live on a low-dimensional simplex and are fitted on hundreds of samples, so the selection cannot meaningfully overfit; the following proposition quantifies this with explicit constants.

Proposition 4.3 (validation stacking). *Let $f_1, \dots, f_M : \mathcal{U} \rightarrow \mathcal{V}$ be fixed maps, let $\Delta = \{w \in \mathbb{R}^M : w_m \geq 0, \sum_m w_m = 1\}$, and for $w \in \Delta$ write $F_w = \sum_m w_m f_m$. Assume the members' per-sample relative errors are bounded: $\ell(f_m(u), G(u)) \leq B$ for μ -a.e. u and every m .*

- (i) *For every $w \in \Delta$, $\ell(F_w(u), G(u)) \leq \sum_m w_m \ell(f_m(u), G(u))$ pointwise; in particular $R(F_w) \leq \sum_m w_m R(f_m) \leq \max_m R(f_m)$, and $\ell(F_w(u), G(u)) \leq B$.*
- (ii) *Let $\hat{w}$ minimize the empirical risk $\hat{R}(w) = \frac{1}{m} \sum_{i=1}^m \ell(F_w(\tilde{u}_i), G(\tilde{u}_i))$ over Δ , computed on a validation sample $\tilde{u}_1, \dots, \tilde{u}_m \sim \mu$ independent of $f_1, \dots, f_M$. Then for every $\delta \in (0, 1)$, with probability at least $1 - \delta$,*

$$R(F_{\hat{w}}) \leq \min_{w \in \Delta} R(F_w) + \frac{8B}{m} + B \sqrt{\frac{2((M-1) \log(m(M-1)+1) + \log(2/\delta))}{m}}.$$

With $M = 4$ members, $m = 1000$ validation samples and $\delta = 0.05$ the right-hand excess evaluates to about $0.24 B$; at $B = 0.25$, the largest per-sample member error we observe, this caps the selection cost at roughly six percentage points. The bound is conservative, as such bounds are: the realized validation-to-test gap of the stack in Section 5 is two orders of magnitude smaller. Its role is the scaling $\sqrt{M \log m/m}$, which says that fitting a handful of convex weights on a thousand held-out samples is statistically almost free; this is why the pipeline spends its validation data there and on the kernel hyperparameters and nowhere else.

4.4 What the stack can achieve

Proposition 4.3 says fitting the weights is safe; it does not say mixing is worth anything. That is decided by a single measurable matrix. For a map f write the *normalized residual* at u as $\rho_f(u) = (f(u) - G(u)) / \|G(u)\|_2$, so that $\ell(f(u), G(u)) = \|\rho_f(u)\|_2$.

Proposition 4.4 (second-moment identity for convex stacks). *Let $f_1, \dots, f_M$ be fixed maps and define $S \in \mathbb{R}^{M \times M}$ by $S_{mk} = \mathbb{E}_{u \sim \mu} \langle \rho_{f_m}(u), \rho_{f_k}(u) \rangle$. For w in the simplex Δ^{M-1} let $f_w = \sum_m w_m f_m$. Then*

$$\mathbb{E}_{u \sim \mu} \ell(f_w(u), G(u))^2 = w^\top S w,$$

so the squared-metric risk of every convex stack is determined by S , and the best achievable is $\min_{w \in \Delta^{M-1}} w^\top S w$. In particular:

- (i) *(two members) if S has diagonal (e_1^2, e_2^2) with $e_1 \leq e_2$ and correlation $\varrho = S_{12}/(e_1 e_2)$, mixing in the weaker member strictly helps if and only if $\varrho < e_1/e_2$, in which case the optimum is interior with value $e_1^2 e_2^2 (1 - \varrho^2) / (e_1^2 + e_2^2 - 2\varrho e_1 e_2)$;*
- (ii) *(equicorrelated family) if $S_{mm} = e^2$ and $S_{mk} = \varrho e^2$ for all $m \neq k$ with $\varrho \geq 0$, then $\min_w w^\top S w = e^2 (\varrho + (1 - \varrho)/M)$, attained at uniform weights and decreasing to $e^2 \varrho$ as $M \rightarrow \infty$.*

Part (ii) is a floor: no number of additional members from an equicorrelated family moves the stack's root-mean-square relative error below $e\sqrt{\varrho}$. The reported metric is the mean rather than the root mean square of $\|\rho_f\|_2$, and the two differ by the dispersion of the per-sample error; across every pipeline in this paper their ratio stays within a few percent of 0.93, so predictions made through S transfer to the reported metric essentially unchanged. The proposition frames

the diversity experiment of Section 5.3: what a new member is worth is legible in the row it adds to S , before any weights are fitted. On this benchmark the rows all look alike, pairwise correlations of 0.86–0.96 among trained networks of three architecture families and two losses, 0.79–0.89 against the kernel predictor, so the floor sits just below the best single member, and the measured stack lands on it. Part (i) is the same computation specialized to two members; it correctly predicts, from measured (e_1, e_2, ϱ) alone, which members receive weight in the fitted stack and which are dropped. Section 6 applies the same test on a problem where the answer comes out the other way: there the kernel’s error is inflated by a factor the next subsection quantifies, the threshold e_1/e_2 collapses, and the kernel is dropped even at a residual correlation of 0.14.

4.5 Metric and representation: when a fixed kernel loses to a network

The structural-mechanics benchmark has the neural mean and the isotropic kernel within half a point of each other. On the OCO-2 emulation problem of Section 6 the same fixed kernel is an order of magnitude behind the same network. Two mechanisms can produce such a gap, and they separate cleanly.

Suppose the operator factors as $G(u) = g(Au)$ with $A \in \mathbb{R}^{d \times d}$ nonsingular and g of unit H^s norm, and that μ has a density bounded above and below on a compact domain. The singular values $\sigma_1 \geq \dots \geq \sigma_d$ of A are the rates at which G varies along the input directions; write r for the number carrying a fixed fraction of $\sum_i \sigma_i^2$.

Proposition 4.5 (isotropic and adapted kernel rates). *Let $\hat{G}_n^{\text{iso}}$ be kernel ridge regression with a Matérn kernel of smoothness s and a validation-optimal single length scale, and $\hat{G}_n^{\text{ard}}$ the same construction in the metric $u \mapsto Au$. With high probability over n i.i.d. samples,*

$$\mathbb{E} \|G - \hat{G}_n^{\text{iso}}\| \leq C \sigma_1^s n^{-s/d}, \quad \mathbb{E} \|G - \hat{G}_n^{\text{ard}}\| \leq C n^{-s/r},$$

with C depending only on g , s and the density bounds; the ratio of the two bounds is $\sigma_1^s n^{s(1/r-1/d)}$.

The proof (Appendix A) is the scattered-data estimate $\|G - \hat{G}_n\| \lesssim h_X^s \|G\|_{H^s}$ applied to the two designs: one length scale must fill all d directions, so $h_X \asymp n^{-1/d}$ and the norm carries σ_1^s ; the adapted metric confines the variation to r directions and removes the anisotropy from the norm. The proposition bounds the *metric* part of a network’s advantage, the part an anisotropic kernel would recover. The rest is *representational*: a stationary kernel of fixed smoothness reaches only its native space, at any metric, while the network adapts its features to the map. The two parts are separated experimentally by handing the kernel the anisotropic metric and seeing how much of the gap closes; on the OCO-2 band of Section 6 the metric accounts for a factor of about 1.4 of a tenfold gap and representation for the rest, and the direct proof of the representational reading is that the same kernel machinery applied to the network’s learned features recovers the network’s accuracy and slightly exceeds it.

4.6 An error bound for the residual kernel correction

The final stage of the pipeline is kernel ridge regression of the ensemble residual. The bound below is a vector-valued, residual form of the classical power-function estimate for kernel interpolation [20] and of the optimal-recovery viewpoint of Owhadi and Scovel [15]; we include the short argument in Appendix A to keep the two factors explicit. Fix the mean $m : \mathcal{U} \rightarrow \mathcal{V}$ (in the

pipeline, the stacked ensemble; in the statement below m is any fixed map independent of the correction sample) and write $r = G - m$ for the residual operator with components r_j , $j = 1, \dots, q$. Let k be a positive definite kernel on $\mathcal{U}$ with RKHS $\mathcal{H}_k$, and assume $r_j \in \mathcal{H}_k$ for all j with

$$\|r\|_K^2 := \sum_{j=1}^q \|r_j\|_{\mathcal{H}_k}^2 < \infty.$$

Given inputs $X = (u_1, \dots, u_n)$, Gram matrix $K = k(X, X)$ and nugget $\lambda \geq 0$, the correction is $\hat{r}_\lambda(u) = k(u, X) (K + n\lambda I)^{-1} R$, $R_{nj} = r_j(u_n)$, and the corrected predictor is $m + \hat{r}_\lambda$. Let

$$P_\lambda(u)^2 = k(u, u) - k(u, X) (K + n\lambda I)^{-1} k(X, u) \quad (2)$$

denote the Gaussian process posterior variance with the same nugget (for $\lambda = 0$ this is the classical power function of the point set X).

Theorem 4.6 (certified pointwise bound). *For every $u \in \mathcal{U}$ and every $\lambda \geq 0$,*

$$\|G(u) - m(u) - \hat{r}_\lambda(u)\|_2 \leq \|G - m\|_K \tilde{P}_\lambda(u) \leq \|G - m\|_K P_\lambda(u),$$

where $\tilde{P}_\lambda(u)^2 = P_\lambda(u)^2 - n\lambda \|(K + n\lambda I)^{-1} k(X, u)\|_2^2$.

Three comments. First, the inequality is algebraic: it holds pathwise for every fixed m , every dataset, and every u , with no probabilistic assumptions. In the pipeline m is itself fit on the same training set; this does not affect the validity of the bound, only the reading of $\|G - m\|_K$ as a random (realized) quantity rather than an a priori one. Second, the bound factorizes into a quantity that depends only on the residual operator ($\|G - m\|_K$) and a quantity that depends only on the kernel and the design (P_λ), and the second factor is exactly the posterior standard deviation that a Gaussian process interpretation of the correction would report. This is the sense in which the bound is certified: whatever error the corrected surrogate commits at u is controlled by the reported $P_\lambda(u)$ times a constant that does not depend on u . We stress that this is a statement about a valid upper bound, not about the usefulness of P_λ as a pointwise error *ranking*. Section 5.4 finds that on this benchmark P_λ ranks the error poorly, because the neural mean supplies most of the error and the relative metric is confounded by output amplitude; a distribution-free conformal rescaling of P_λ nonetheless attains its nominal coverage on the test set. Third, the theorem quantifies why one should regress residuals rather than raw targets: the design factor P_λ is the same in both cases, so the gain of the neural mean is the drop from $\|G - \bar{v}\|_K$ (kernel-only, mean $\bar{v}$) to $\|G - m\|_K$. These norms are not directly observable, but the RKHS norms of the *fitted* interpolants, $\|\hat{r}_\lambda\|_K^2 = \text{tr}(\alpha^\top K \alpha)$ with $\alpha = (K + n\lambda I)^{-1} R$, are computable lower-bound proxies, and they drop by a factor of about 271 when the kernel is moved from raw targets to ensemble residuals (Table 5). The neural mean does not merely reduce the size of the residual in L^2 ; it leaves behind a residual operator that is genuinely smoother as seen by the kernel.

4.7 Distribution-free coverage for the reported band

Theorem 4.6 controls the error by $\|G - m\|_K P_\lambda(u)$, but the constant $\|G - m\|_K$ is not known a priori, and Section 5.4 shows that P_λ alone ranks the error weakly. What the surrogate reports as

uncertainty is therefore not P_λ itself but a rescaling of it, $q P_\lambda(u)$, whose multiplier q is calibrated on the held-out split by the split-conformal rule

$$q = Q_{1-\alpha} \left(\{s_i := \|e(\tilde{u}_i)\|_2 / P_\lambda(\tilde{u}_i)\}_{i=1}^m \right), \quad e(u) = G(u) - m(u) - \hat{r}_\lambda(u), \quad (3)$$

where $Q_{1-\alpha}$ is the $\lceil (1-\alpha)(m+1) \rceil$ -th smallest value of the m validation scores. The point of this construction is that its coverage needs neither the bound to be tight nor P_λ to be a good error ranking; it needs only exchangeability, which the fixed splits provide.

Proposition 4.7 (finite-sample coverage). *Fix the mean m , the correction $\hat{r}_\lambda$ and the design X , and let $P_\lambda(\cdot) > 0$. Suppose the validation and test inputs $\tilde{u}_1, \dots, \tilde{u}_m, u^*$ are exchangeable (in particular i.i.d. from μ) and drawn independently of $m, \hat{r}_\lambda, X$. Let q be the conformal multiplier (3) and define the band $C(u) = \{v : \|v - m(u) - \hat{r}_\lambda(u)\|_2 \leq q P_\lambda(u)\}$. Then*

$$1 - \alpha \leq \mathbb{P}(G(u^*) \in C(u^*)) \leq 1 - \alpha + \frac{1}{m+1},$$

the upper bound holding when the scores are almost surely distinct.

The two-sided statement is the standard guarantee of split conformal prediction [10, 19], transported to the functional-output setting by taking the nonconformity score to be the relative residual norm $s = \|e(u)\|_2 / P_\lambda(u)$; the proof, a rank argument on the exchangeable scores, is in Appendix A. Three points make it the right closing statement for the uncertainty analysis. It is exactly the quantity we compute: the reported coverage of 91.6% at the nominal $1 - \alpha = 90\%$ in Section 5.4 realizes this proposition with $m = 1000$ and $\alpha = 0.1$, and the slack $1/(m+1) \approx 0.001$ explains why the empirical coverage sits just above nominal rather than exactly on it. It is agnostic to everything the earlier results leave uncertain: the neural mean may be biased, P_λ may correlate with the error weakly or with the wrong sign, and the bound of Theorem 4.6 may be loose, yet the band still covers at the stated rate. And it is where the uncertainty analysis ends: among the candidate uncertainty signals we examine, the conformally rescaled P_λ is the only one that carries a guarantee, so it is the one the surrogate reports.

4.8 Effective dimension from the Gram spectrum

The remaining design choice is the nugget λ , selected by cross-validation in the pipeline. The following exact identity connects that choice to the spectrum of the Gram matrix and, through it, to random matrix descriptions of the design.

Lemma 4.8 (effective dimension and the empirical Stieltjes transform). *Let K be a symmetric positive semidefinite $n \times n$ matrix with eigenvalues $\lambda_1, \dots, \lambda_n \geq 0$, let $\hat{m}(z) = \frac{1}{n} \sum_{i=1}^n (\lambda_i / (n - z))^{-1}$ be the Stieltjes transform of the empirical spectral distribution of K/n . Then for every $\lambda > 0$ the effective dimension of ridge regression with nugget λ satisfies*

$$d_{\text{eff}}(\lambda) := \text{tr}(K(K + n\lambda I)^{-1}) = n(1 - \lambda \hat{m}(-\lambda)).$$

In particular, if the empirical spectral distribution of K/n converges weakly to a limit law F as $n \rightarrow \infty$, then $d_{\text{eff}}(\lambda)/n \rightarrow 1 - \lambda m_F(-\lambda)$ with m_F the Stieltjes transform of F .

The identity is unconditional; the random-matrix content enters through the choice of F . For the benchmark's simplest reference model, isotropic linear features, the limit can be evaluated in closed form.

Corollary 4.9 (effective dimension under a Marčenko–Pastur limit). *Let $K = XX^\top$ with $X \in \mathbb{R}^{n \times p}$ having i.i.d. entries of mean zero and variance σ^2 , and let $p/n \rightarrow \gamma \in (0, 1]$. Then*

$$\frac{d_{\text{eff}}(\lambda)}{n} \rightarrow \gamma(1 - \lambda m(-\lambda)), \quad m(-\lambda) = \frac{\sqrt{(\lambda + \sigma^2(1 - \gamma))^2 + 4\gamma\sigma^2\lambda} - (\lambda + \sigma^2(1 - \gamma))}{2\gamma\sigma^2\lambda},$$

almost surely, where m is the Stieltjes transform of the Marčenko–Pastur law with ratio γ and scale σ^2 . In a simulation with $n = 6000$, $p = 1800$, $\sigma^2 = 1.7$, the formula agrees with the empirical $d_{\text{eff}}(\lambda)/n$ to four decimal places across $\lambda \in [10^{-3}, 10]$.

Two uses. Quantitatively, the formula makes the nugget–capacity trade-off explicit for the bulk of a feature Gram matrix: eigenvalue mass of Marčenko–Pastur type is absorbed or discarded by λ according to a single closed-form curve, and outlying (spiked) eigenvalues x_s simply add their terms $x_s/(x_s + n\lambda)$ on top. Qualitatively, it separates the two regimes visible in Figure 6: the Matérn Gram matrix on the loads is nothing like a Marčenko–Pastur bulk (its spectrum decays exponentially, which is why d_{eff} is a few hundred out of 19000), whereas the Gram matrices of learned penultimate features do show a bulk-plus-spikes shape, and for them the corollary describes how much of the bulk the cross-validated nugget retains. For the feature Gram matrices of the trained networks we observe the familiar picture of a bulk that is well fitted by a Marčenko–Pastur law [13] together with a small number of outlying eigenvalues carrying the regression signal, as in spiked covariance models [1]; for the Matérn Gram matrix on the 41-dimensional loads the spectrum decays rapidly and d_{eff} is small ($\approx$ a few hundred at the cross-validated λ , out of $n = 19000$; Figure 6). This gives a post-hoc reading of the cross-validated nugget: λ lands where $d_{\text{eff}}(\lambda)$ has absorbed the outlying eigenvalues and the leading bulk, and further decreasing λ buys capacity precisely where the spectrum carries little signal. It also explains why the exact solve is affordable: the correction is numerically a problem of dimension d_{eff} , not n .

The effective dimension is moreover exactly the total budget of the design factor from Theorem 4.6. Writing $A = K + n\lambda I$, the posterior variances at the training inputs sum to

$$\sum_{i=1}^n P_\lambda(u_i)^2 = \text{tr}(K) - \text{tr}(KA^{-1}K) = \text{tr}(n\lambda KA^{-1}) = n\lambda d_{\text{eff}}(\lambda),$$

using $KA^{-1}K = K - n\lambda KA^{-1}$. So the same d_{eff} that reads the nugget off the spectrum also fixes, up to the factor $n\lambda$, the total posterior-variance mass the correction distributes over the design; the two halves of this section are one quantity seen from two sides.

5 Results on the structural-mechanics benchmark

5.1 Main comparison

Table 1 reports the high-data protocol. The published numbers are quoted from de Hoop et al. [5] (as tabulated by Batlle et al. [2]) and from Batlle et al. [2]; our rows are computed under the split of Section 2, which gives our methods strictly no more training information than the baselines had. The kernel ridge row reproduces the published kernel result almost exactly (5.19% against 5.18%), which we take as evidence that the protocols are aligned; the gap opened by the rest of the pipeline is therefore not an artifact of evaluation choices. The full pipeline reaches

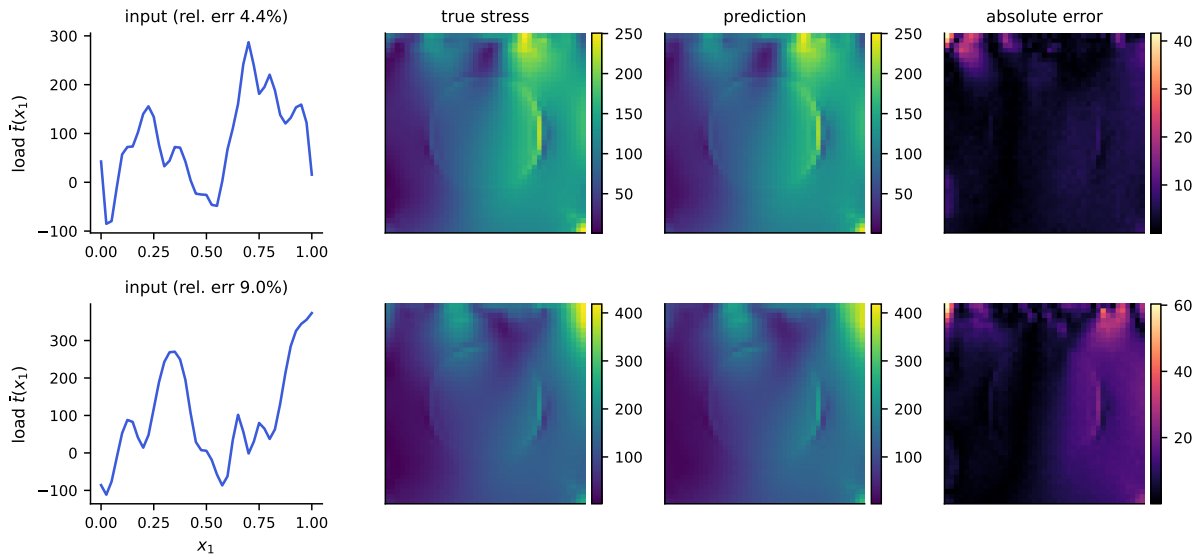


Figure 1: Example predictions of the full pipeline. Top: a median-error test case; bottom: a case at the 98th error percentile. Columns: input load, true von Mises stress, prediction, and pointwise absolute error (note the smaller scale). Errors concentrate in the high-traction corner, as also reported by Mora et al. [14].

4.55%. This is below every published method other than PARA-Net (4.55%), which it matches at the reported two-decimal precision; since our own stacking configurations vary by a hundredth or two among themselves (the ablation below moves between 4.65% and 4.67% under changes that should not matter), we read the two as tied and do not claim to separate them. The margin over the FNO (4.76%), PCA-Net (4.67%), DeepONet (5.20%) and the optimal-recovery kernel (5.18%) is several times that spread. Section 5.3 argues the tie is not a coincidence of tuning but a floor the data impose on every method here.

5.2 Ablation

Table 3 builds the pipeline up one component at a time under the high-data protocol. The kernel ridge on loads and the reflection-averaged neural mean start at 5.19% and 4.86% respectively. Stacking is trivial with a single high-data mean and leaves it unchanged; the value of the combination appears in the correction stage, where regressing the neural mean’s residual with the validated Matérn kernel lowers the error to 4.55%. The feature-space and second input-space corrections did not improve validation error beyond the first correction with a single mean and so were not selected; we expect them to contribute more with a larger and more diverse ensemble, and we report the stages that the validation split actually chose. Two smaller effects are worth isolating. Reflection test-time averaging lowers the MLP test error by 0.11 points, consistent with Proposition 4.1, which guarantees the direction of the change. Adding the kernel-flow regularizer (Section 4.2) to the low-data MLP did not help here (it moved test error from 5.64% to 5.87%); Proposition 4.2 says what the term estimates, not that estimating it is useful on every problem, and on this smooth-output benchmark the plain metric loss was already a good proxy. Nor does a richer correction kernel help: replacing the single tuned Matérn with a sum of Matérn kernels at several bandwidths leaves the validation error unchanged to two decimals, which is consistent

Table 1: Structural mechanics, high-data protocol (20000 training samples, 20000 test). Mean relative L^2 test error. Rows above the rule are published results on the same data; rows below are ours (mean over the fixed split; TTA denotes reflection averaging at test time).

Method	Error (%)	Parameters
DeepONet [5]	5.20	–
FNO [5]	4.76	–
PCA-Net [5]	4.67	–
PARA-Net [5]	4.55	–
Optimal-recovery kernel [2]	5.18	–
Kernel ridge on loads (ours)	5.19	–
Residual MLP + reflection TTA	4.86	4.9M
+ residual kernel correction (full pipeline)	4.55	4.9M

Table 2: Low-data protocol (1250 training samples, 20000 test). Published rows from Mora et al. [14]. Our pipeline sees only the 1250 labels (250 of them used as the validation split).

Method	Error (%)
DeepONet	8.70
GP, optimal recovery [2]	6.95
Zero-mean GP [14]	6.74
FNO	6.62
FNO-mean GP [14]	6.49
Kernel ridge on loads (ours, multiscale)	6.66
Full pipeline (ours)	5.38

with the residual being close to kernel-irreducible in the input geometry, and is why the boosted correction stages of Section 3.3 were not selected. One positive lesson does emerge from the refiner variants. Conditioning the refiner on the kernel method’s prediction (test error 4.73%) beats conditioning it on another network’s prediction (4.84%), even though the two conditioning fields have almost the same accuracy; the kernel field carries information complementary to the network’s inductive bias, whereas a second network’s field is largely redundant. Enlarging the ensemble beyond the residual family is the subject of the next subsection, where the returns of each addition turn out to be predictable in advance from the correlation structure of the members’ errors.

5.3 Architectural diversity and the shared residual

The natural reading of Table 1 is that the remaining error is a modeling problem: different architectures make different mistakes, so a more diverse ensemble should stack to a lower number. We tested this directly. To the residual family of Section 3 we added a Fourier neural operator, a UNet, and a variant of the MLP trained on normalized mean squared error instead of the metric, each trained to the published single-model level or better (Table 4). The test refutes the reading. Figure 2 shows the correlation of per-sample normalized residuals between members: every pair

Table 3: Building the high-data pipeline. Mean relative L^2 test error under the 20000-sample protocol; “+” rows are cumulative.

Stage	Test error (%)
Kernel ridge on loads	5.19
Reflection-averaged neural mean	4.86
+ kernel-conditioned refiner, stack	4.67
+ diverse members (Section 5.3), per-pixel stack	4.58
+ residual kernel correction	4.55

of trained networks, across three architecture families and two losses, correlates between 0.86 and 0.96, and the kernel predictor, the most alien member by construction, still correlates 0.79–0.89 with all of them. The members are not making different mistakes. They are making the same mistake with small private variations.

The second-moment identity of Proposition 4.4 makes the consequence quantitative before any weights are fitted. The matrix S measured on the validation split predicts both the optimal convex weights and the achievable stack risk: for the six-member ensemble the predicted root-mean-square relative error of the best mixture is 4.97%, the fitted stack realizes a mean of 4.61%, and their ratio is the dispersion factor ≈ 0.93 that is stable across every pipeline we ran. The weights the prediction assigns from S alone, without ever evaluating the metric, concentrated on the refiner and the FNO with a few tenths each on the UNet and the MSE-trained network, a few percent on the kernel, and zero on the plain MLP whose error is already spanned by the better members of its family, match the weights the direct metric search finds, as part (i) of the proposition predicts from the measured (e_1, e_2, ρ) . Stacking does exactly what the correlations permit, and members like these permit little: part (ii) puts the infinite-ensemble floor of an equicorrelated family at $e\sqrt{\rho}$, which at $e \approx 4.7\%$ and a mean pairwise $\rho \approx 0.9$ is about 4.5%. Allowing the weights to vary over the grid (an affine per-pixel stack, fitted by ridge regression on half the validation split and accepted only because it beat the global weights on the other half) recovers a little of what global weights cannot see, and the kernel correction adds a few hundredths on top; that is the 4.55% of Table 1, and it is consistent with the floor just computed.

Where does the common mistake live? Write r_m for member m ’s residual field on a validation case and $c = \frac{1}{M} \sum_m r_m$ for the across-member mean, the component that no amount of uniform averaging removes. Measured over the validation split and three families (refiner, FNO, kernel), the shared component carries 90% of the average residual energy (Figure 3). Three of its properties matter. Spatially, its energy concentrates at the two corners of the loaded edge, where the traction boundary condition meets the lateral supports; relative to the local field scale it is three to four times larger there than the grid average, and these are precisely the locations where the finite element solution is least accurate and where interpolation to the 41×41 grid is most strained. Spectrally, it is enriched in high radial frequencies relative to the stress fields themselves: the fields put 97.5% of their energy below radial wavenumber 4, the shared residual only 69%. And its magnitude is statistically independent of everything we can compute from the input: the correlation of $\|c\|$ with the output norm is 0.01, with the load norm 0.02, with the load’s total variation 0.02. A component that no architecture avoids, that no input statistic predicts, that lives at the stress concentrations, and that fixed-scale additive noise would reproduce is most economically read as noise of the data-generating process itself, finite element

Table 4: Ensemble members, high-data protocol. Mean relative L^2 test error with reflection averaging; every member is selected on the validation split.

Member	Family / loss	Error (%)
Residual MLP	MLP, metric loss	4.86
Residual MLP	MLP, normalized MSE	4.71
Kernel-conditioned refiner	MLP on $(u, \text{KRR}(u))$	4.73
FNO	spectral, metric loss	4.70
UNet	convolutional, metric loss	4.99
Kernel ridge on loads	Matérn-5/2	5.19
Per-pixel stack (validation-fitted)	–	4.58
+ residual kernel correction	–	4.55

and grid-interpolation error at the singular corners, not as approximation error of the surrogates.

Two independent measurements corroborate this reading. First, the residual kernel correction, which regresses the stack’s residual on the load, improves the stack by only a few hundredths of a point at $n = 19000$ whatever the kernel scale: at this sample size the shared residual is not a function of the load in any way a Matérn RKHS on $\mathbb{R}^{41}$ can see. Second, the error stops responding to data. Under identical recipes, the MLP’s test error is 4.95% with 3500 training samples, 4.86% with 8500, and 4.86% with 19000 (Figure 5); the kernel ridge curve still falls, at roughly $n^{-0.11}$, but toward the same region. A modeling limitation should yield to capacity, diversity, or data. This error yields to none of them.

We draw two conclusions. First, the published plateau of Table 1, five methods within two thirds of a point of each other after years of architectural progress, is not evidence that some sixth architecture is missing; it is the signature of a benchmark whose achievable error is set by its data. On the evidence above, numbers near 4.5% are within a few hundredths of that limit, which is where our pipeline lands (4.55%), and material further progress on this benchmark means regenerating the data (finer meshes near the corners, higher-order elements, output grids that resolve the concentrations), not new surrogates. Second, the practical recipe survives the reinterpretation with its economics clarified: symmetrized accurate means capture what is learnable, stacking buys exactly the decorrelation the members possess (little, here), and the kernel correction certifies and mops up the load-visible remainder. The components are worth their cost in that order.

5.4 Uncertainty quantification

Theorem 4.6 certifies the corrected surrogate’s error by $\|G - m\|_K P_\lambda(u)$, and it is tempting to read the design factor P_λ as a pointwise error bar. On this benchmark that reading fails, in an instructive way. Table 5 confirms the operator factor behaves as the theory predicts: the computable RKHS norm of the fitted correction is about $271\times$ smaller when the kernel regresses ensemble residuals than when it regresses raw stress fields, so an accurate mean genuinely leaves a smoother remainder and a tighter certificate. But the design factor P_λ correlates only weakly with the corrected surrogate’s absolute error (Pearson 0.08), and *negatively* with the benchmark’s relative error. The cause is measurable: P_λ is large for loads far from the training set, those loads tend to have large amplitude, large-amplitude loads produce large-norm stress fields, and

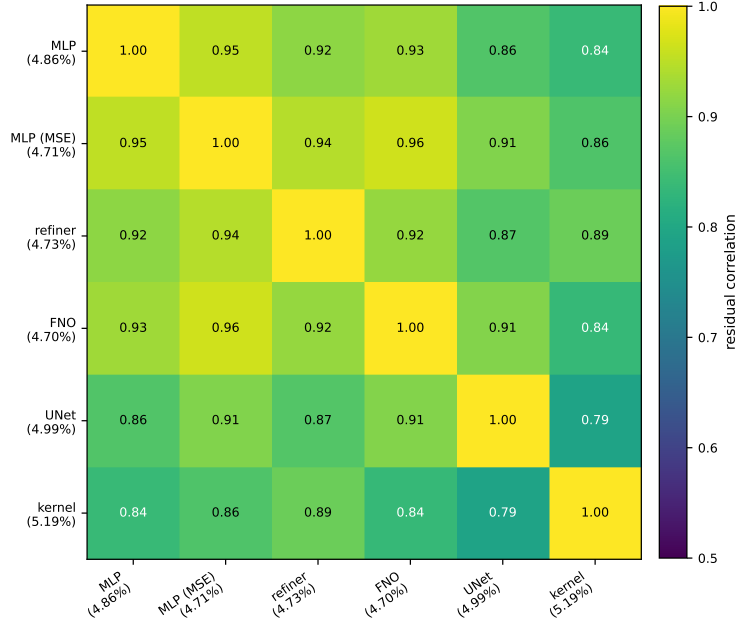


Figure 2: Correlation of per-sample normalized residuals between ensemble members on the validation split (test-set values agree to two decimals). Every pair of trained networks correlates between 0.86 and 0.96, the convolutional UNet being the least like the rest; the kernel predictor, the most alien member by construction, still correlates 0.79–0.89 with all of them.

dividing by the output norm makes their relative error small. Indeed P_λ correlates 0.65 with the output norm (Figure 4). The error is dominated by the neural mean, whose mistakes are not a function of the input-space geometry that P_λ sees, so a purely input-space power function cannot rank them.

The obvious alternative fares no better, and with the ensemble of Section 5.3 the question can be answered rather than left open. Ensemble disagreement, the spread of the members’ predictions, is the standard deep-ensemble error signal; on the full cross-architecture ensemble its correlation with the corrected surrogate’s absolute error is 0.08, no better than the 0.10 of the like-architecture ensemble. Section 5.3 says why it cannot do better here: disagreement measures the member-specific components of the error, which carry under a tenth of the residual energy, while the ranking signal for the shared component is invisible to any within-ensemble statistic, because the members agree precisely where they are jointly wrong.

What does hold is coverage, for free. A distribution-free conformal rescaling, taking the 0.9-quantile of $\|e\|_2/P_\lambda$ on the validation split and scaling P_λ by it, yields a band with 91.6% empirical coverage on the test set at the nominal 90% level. Conformal validity needs no correlation between the score and the error, only exchangeability, so it survives the confound that defeats the ranking. The lesson is narrow but real: a valid pointwise bound (Theorem 4.6) is not automatically a useful pointwise indicator, the standard uncertainty signals can both fail on the same problem, and a relative-error metric has to have its normalization accounted for before a posterior standard deviation means what it appears to.

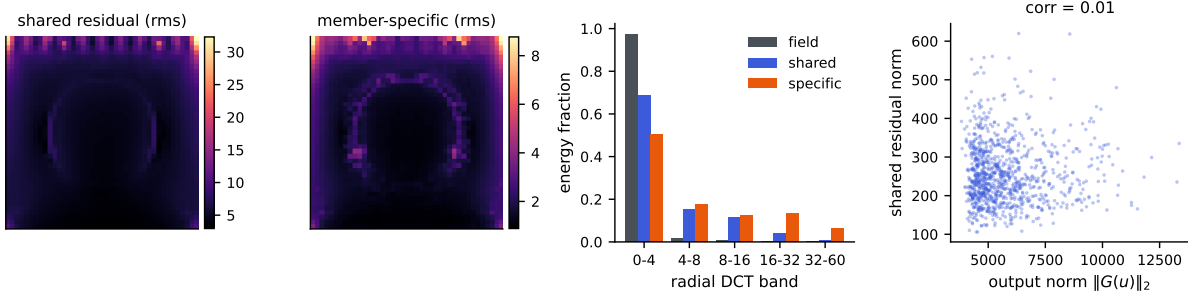


Figure 3: Anatomy of the shared residual c (across-member mean of the residual fields; refiner, FNO and kernel members). Left to right: rms of c over the grid; rms of the member-specific remainders; radial DCT band energies of the stress fields, of c , and of the specific parts; and the norm of c against the output norm, whose correlation is 0.01.

Table 5: Squared RKHS norm of the fitted kernel interpolant, $\text{tr}(\alpha^\top K \alpha)$, when the same validated kernel and design regress raw stress fields versus ensemble residuals.

Kernel regresses	$\ \hat{r}\ _K^2$
Raw stress fields ($m = 0$)	1.39×10^{10}
Ensemble residuals	5.14×10^7
Ratio	$271 \times$

5.5 Spectra and effective dimension

Figure 6 shows the spectrum of the Matérn Gram matrix on the loads and the effective dimension $d_{\text{eff}}(\lambda)$ computed from it by the exact identity of Lemma 4.8. The spectrum decays quickly: a few hundred eigenvalues carry essentially all the mass, the tail falling many orders of magnitude below the leading modes. At the cross-validated nugget the effective dimension is a few hundred out of $n = 19000$, which is why the exact Cholesky solve is affordable and why the correction generalizes despite interpolating nineteen thousand points; the problem the kernel actually solves has the dimension of the retained spectrum, not of the sample. The cross-validated λ sits at the shoulder of the d_{eff} curve, past the leading modes and into the rapidly decaying tail, matching the reading of Lemma 4.8.

5.6 Cost and reproducibility

The pipeline was built and run on a single laptop. The kernel stages are exact: one Cholesky factorization of the 19000×19000 Matérn Gram matrix in double precision takes about a minute on a desktop CPU, and prediction is two matrix products; there are no inducing points, stochastic solvers, or preconditioners, and no GPU is needed for the correction. The neural means are small by the standards of the literature (a few million parameters) and train in the metric itself. The exactness is affordable precisely because, as Section 5.5 shows, the effective dimension of the solve is a few hundred: the Gram matrix is numerically low rank at the working nugget, so the factorization does far less work than its nominal $O(n^3)$ suggests. All splits are fixed by a published permutation seed, the input is consumed as the 41-dimensional load after the exact

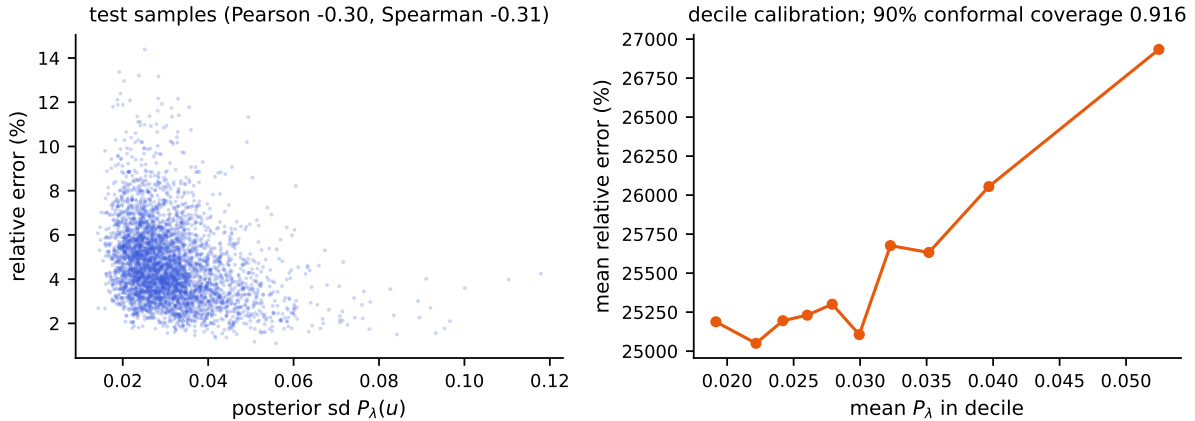


Figure 4: Left: the Gaussian process posterior standard deviation P_λ against the corrected surrogate’s relative error on the test set; the association is weak and, because of the output-amplitude confound, slightly negative. Right: decile calibration of P_λ against absolute error, the theory-consistent quantity. A distribution-free conformal rescaling of P_λ attains 91.6% coverage at the nominal 90% level.

broadcast check of Section 2, and every model is selected on the validation split and evaluated once on the test set; we release the splits, the trained means, and the code so the table can be regenerated end to end.

6 The OCO-2 radiative-transfer emulator

The structural-mechanics benchmark put the neural mean and the kernel in the balanced regime. To see the other regime we apply the same components to the radiative-transfer emulation problem of Lamminpää et al. [9]: per spectral band of the OCO-2 instrument, learn the map from a reduced atmospheric state (20–24 dimensions after the dimension reduction of that paper) to the reduced radiance (40 PCA coefficients of the monochromatic spectrum), from 20000 training pairs. The data, and the kernel-flow emulator’s own predictions on the 2000 test states, are public (OSF project [u2t8a](#)), so the comparison below is computed on identical test points against the published emulator itself rather than against our reimplement of it.

Two error metrics matter, and they disagree in an instructive way. The *reduced* metric is the relative L^2 error on the 40 standardized coefficients. The *radiance* metric maps predictions back to the monochromatic spectrum through the stored PCA projection and norms before comparing; because the projection is orthogonal, on the reduced side it is exactly the diagonal weighting s_z , and its energy concentrates almost entirely on the first few coefficients.

Table 6 carries three findings. First, the representation ladder: the same exact kernel machinery scores 40% on the raw input, 30% after rescaling each input coordinate by its measured sensitivity (the metric term of Proposition 4.5: the effective rank here is 17 of 20, so the proposition predicts a constant-factor gain and no change of rate, which is what the scaling experiment shows), and 3.82% on the trained network’s features, past the network’s own head. The raw-input kernel was limited by its features, not by anything kernel-shaped, and the deep kernel head, an exact solve on learned features, is the strongest single model on the reduced metric.

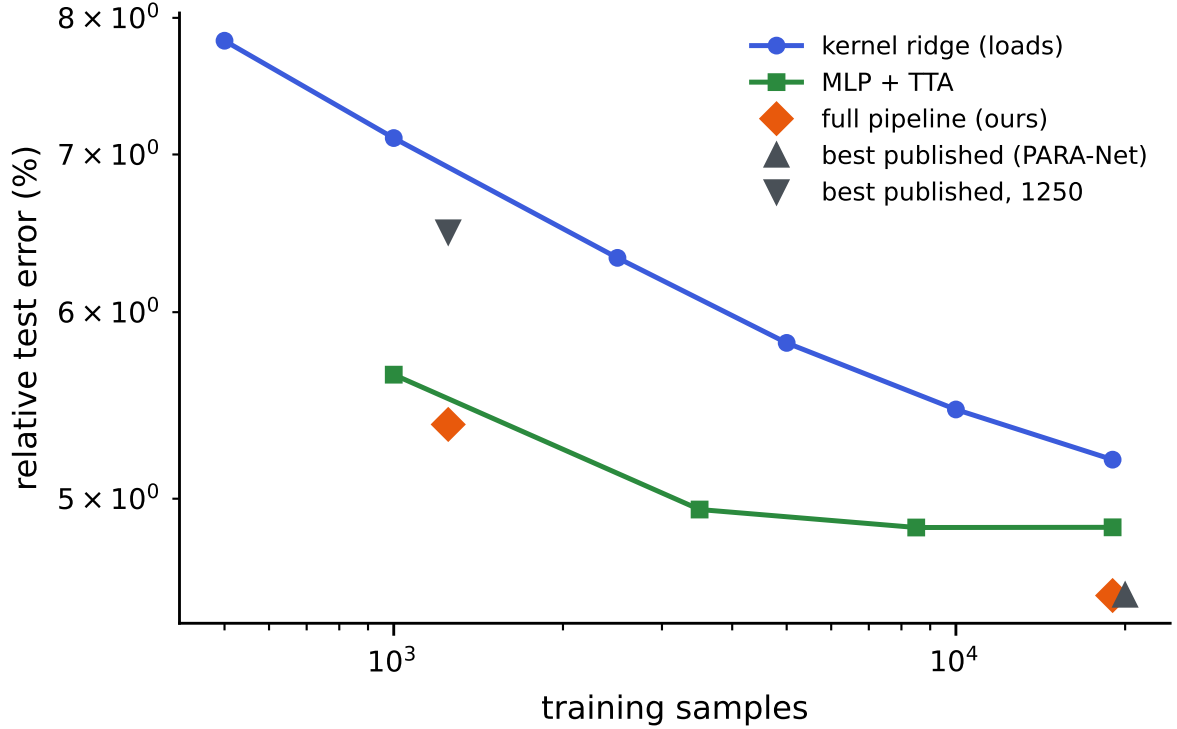


Figure 5: Relative test error against training set size for the kernel ridge on loads (log-log), with the full pipeline and the best published results marked in both regimes.

Second, the training metric behaves as a budget. The flat-trained and radiance-trained rows are one architecture and two losses, and each wins the metric it was trained in by a factor of five or more while losing the other. The mechanism is visible per coordinate: the radiance metric concentrates on the leading coefficient, the kernel-flow emulator (whose lengthscales are tuned per output) predicts that coefficient to 0.0012 root mean square against the flat network’s 0.0054, while the flat network is three to four times more accurate on the trailing thirty coordinates that the radiance barely sees. Training the network in the radiance metric closes exactly that gap. A constant-elasticity model of this reallocation is too crude — the per-coordinate exponents scatter widely, and coordinate errors do not decouple because the network shares capacity — but the direction is robust, and the practical rule is plain: train in the metric you report.

Third, the regimes of Section 4.4 are decided by these numbers before any stack is fit. Against the flat network the raw-input kernel has $\varrho = 0.14$ and $e_n/e_k = 0.12$: the condition of Proposition 4.4 fails and the kernel is dropped, in contrast to structural mechanics where the same test keeps it. Because the radiance metric is diagonal in the reduced coordinates, selecting the best model per coordinate on the validation split is optimal for both metrics simultaneously; that per-coordinate combination of the flat and weighted models is the final surrogate, and it beats the kernel-flow emulator on both metrics at once (3.82% against 16.89% reduced, 0.028% against 0.045% radiance). The second band behaves the same way (the weighted mean reaches 0.037% radiance against the emulator’s 0.060% on WCO2), and the code releases the per-band numbers alongside this paper.

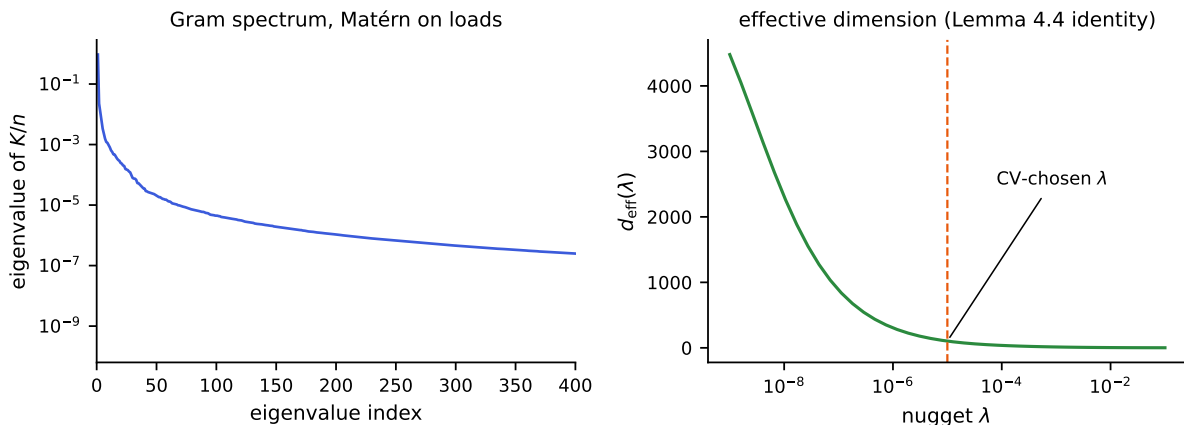


Figure 6: Left: eigenvalues of the Matérn Gram matrix K/n on the loads (log scale); the spectrum decays by many orders of magnitude within a few hundred modes. Right: the effective dimension $d_{\text{eff}}(\lambda)$ from the exact identity of Lemma 4.8, with the cross-validated nugget marked.

Table 6: OCO-2 O2 band, 2000 held-out states, identical test points. The kernel-flow row is the emulator of Lamminpää et al. [9], scored from its own stored predictions. “Features” means the penultimate activations of the trained network; the kernel head is the same exact Matérn solve as everywhere else in this paper.

model	reduced	radiance
Matérn kernel, raw input, one length scale	40.47%	0.240%
Matérn kernel, raw input, sensitivity-scaled (ARD)	30.01%	—
kernel-flow emulator [9]	16.89%	0.0448%
residual MLP, flat metric	3.99%	0.159%
residual MLP, radiance metric	46.3%	0.0291%
Matérn head on the flat network’s features	3.82%	0.0758%
Matérn head on the weighted network’s features	20.0%	0.0282%

7 Discussion

What moved the number, and what stopped it. The improvement over the published band decomposes unevenly. Accurate neural means trained on the reported metric and averaged over the reflection symmetry do most of the work; stacking members from different architecture families adds exactly what their measured residual correlations permit, about a tenth of a point here; the kernel corrections contribute a few hundredths more and, with them, the certificate of Theorem 4.6 and the calibrated band of Proposition 4.7. We did not find evidence that any published architecture was mis-tuned by its authors; the components compose, and the composition had not been tried on this benchmark. What stops further movement is not a missing architecture. Section 5.3 locates the remaining error in a component that all families share, that concentrates where the finite element data are least reliable, that no input statistic predicts, and that neither capacity, nor diversity, nor more data reduces. The margin over the published methods other than PARA-Net is real, and the match with PARA-Net is exact to within run-to-run noise, but both should be read for what they are: the last few learnable

hundredths above a floor set by the data, not a step on the way to zero.

Relation to neural-mean Gaussian processes. Our correction stage is closest in spirit to Mora et al. [14], who place a neural operator inside the mean of a Gaussian process and fit both jointly. The differences are operational but they matter at this scale: we fit the mean first and the kernel after (so the expensive stage is ordinary network training), we tune the kernel by validation rather than by marginal likelihood, we correct with an exact solve at $n = 19000$ rather than train through the kernel, and we add the feature-space second stage. The theory in Section 4 applies verbatim to their setting as well; the measured drop in the fitted RKHS norm (Table 5) is the quantitative reason residual corrections of accurate means are the right place to spend kernel capacity.

Relation to kernel emulation practice. The pipeline is consistent with the experience of the kernel-flow line of work in emulation at scale. The closest instance is the forward-model emulator of Lamminpää et al. [9] for the OCO-2 CO₂ retrievals, in which a Gaussian process with a cross-validation-learned kernel replaces the full-physics radiative transfer code within measurement-error precision; kernel flows have likewise been used to infer convective-storm structure from passive microwave observations [17]. The lesson of that work matches ours: kernels with data-adapted hyperparameters are extremely effective once the input is presented in its physical parametrization and the target has been reduced to something smooth. Here the reduction is performed by the neural ensemble rather than by physics, and the kernel-flow loss itself admits the leave-half-out reading of Proposition 4.2. The OCO-2 setting is also the natural next test bed for the residual coupling studied here. Its training pairs are simulator-generated state-to-radiance maps of exactly the shape this paper exploits, a low-dimensional physical state mapped to a smooth high-dimensional output, the mission’s Level 1 and Level 2 products are publicly distributed through NASA’s Earthdata archive, and the emulation setup of Lamminpää et al. [9] specifies the state parametrization and sampling design, so a neural mean with a validated kernel correction and a conformal wrapper can be evaluated there against the pure-kernel emulator without new data collection.

Limitations. The benchmark has a one-dimensional input function and a fixed geometry; the exact kernel solve exploits both, and problems with high-dimensional input fields would require the usual approximations (inducing points, random features) at some cost to the certificates. The certified bound of Theorem 4.6 controls the error relative to the RKHS norm of the residual operator, a quantity we can only lower-bound empirically; the conformal calibration we report is the honest, assumption-light complement. Training the means on the metric, the reflection steps, and the stacking are benchmark-agnostic, but the specific error levels reported here should be read as properties of this dataset and protocol. Finally, our low-data protocol reproduces that of Mora et al. [14] up to the unavoidable ambiguity of which 1250 samples are used; we use the first 1250 of the training block and report the same 20000-sample test set, and we release splits and code so the comparison can be audited.

Outlook. Three directions seem worth pursuing. First, the benchmark itself: regenerating the dataset with refined meshes at the corner singularities, higher-order elements, or output grids that resolve the concentrations would move the floor that Section 5.3 measures, and would return the problem to being a test of surrogates rather than of its own data. The diagnostic protocol

used there, cross-family residual correlation, the second-moment prediction of Proposition 4.4, and the scaling-in- n check, is cheap to run on any benchmark suspected of the same condition. Second, the same recipe on the remaining benchmarks of de Hoop et al. [5] and Batlle et al. [2], where the input functions are genuinely high-dimensional and the interplay between neural means and kernel corrections should look different. Third, the uncertainty side: P_λ is a design quantity, so it can be optimized, and the connection of Lemma 4.8 between the nugget, the spectrum, and the effective dimension suggests principled ways to spend a fixed computational budget on the correction stage.

References

- [1] J. Baik, G. Ben Arous, and S. Péché. Phase transition of the largest eigenvalue for nonnull complex sample covariance matrices. *Annals of Probability*, 33(5):1643–1697, 2005.
- [2] P. Batlle, M. Darcy, B. Hosseini, and H. Owhadi. Kernel methods are competitive for operator learning. *Journal of Computational Physics*, 496:112549, 2024.
- [3] A. Caponnetto and E. De Vito. Optimal rates for the regularized least-squares algorithm. *Foundations of Computational Mathematics*, 7(3):331–368, 2007.
- [4] M. Darcy, B. Hamzi, G. Livieri, H. Owhadi, and P. Tavallali. One-shot learning of stochastic differential equations with data adapted kernels. *Physica D: Nonlinear Phenomena*, 444: 133583, 2023.
- [5] M. V. de Hoop, D. Z. Huang, E. Qian, and A. M. Stuart. The cost-accuracy trade-off in operator learning with neural networks. *Journal of Machine Learning*, 1(3):299–341, 2022.
- [6] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby. An image is worth 16x16 words: transformers for image recognition at scale. In *International Conference on Learning Representations*, 2021.
- [7] V. Koltchinskii and E. Giné. Random matrix approximation of spectra of integral operators. *Bernoulli*, 6(1):113–167, 2000.
- [8] N. Kovachki, Z. Li, B. Liu, K. Azizzadenesheli, K. Bhattacharya, A. Stuart, and A. Anandkumar. Neural operator: learning maps between function spaces with applications to PDEs. *Journal of Machine Learning Research*, 24(89):1–97, 2023.
- [9] O. Lamminpää, J. Susiluoto, J. Hobbs, J. McDuffie, A. Braverman, and H. Owhadi. Forward model emulator for atmospheric radiative transfer using Gaussian processes and cross validation. *Atmospheric Measurement Techniques*, 18:673–694, 2025.
- [10] J. Lei, M. G’Sell, A. Rinaldo, R. J. Tibshirani, and L. Wasserman. Distribution-free predictive inference for regression. *Journal of the American Statistical Association*, 113(523): 1094–1111, 2018.
- [11] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, and A. Anandkumar. Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations*, 2021.

- [12] L. Lu, P. Jin, G. Pang, Z. Zhang, and G. E. Karniadakis. Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, 3:218–229, 2021.
- [13] V. A. Marčenko and L. A. Pastur. Distribution of eigenvalues for some sets of random matrices. *Mathematics of the USSR-Sbornik*, 1(4):457–483, 1967.
- [14] C. Mora, A. Yousefpour, S. Hosseinmardi, H. Owhadi, and R. Bostanabad. Operator learning with Gaussian processes. *Computer Methods in Applied Mechanics and Engineering*, 434: 117581, 2025.
- [15] H. Owhadi and C. Scovel. *Operator-Adapted Wavelets, Fast Solvers, and Numerical Homogenization*. Cambridge University Press, 2019.
- [16] H. Owhadi and G. R. Yoo. Kernel flows: from learning kernels from data into the abyss. *Journal of Computational Physics*, 389:22–47, 2019.
- [17] S. Prasanth, Z. S. Haddad, J. Susiluoto, A. J. Braverman, H. Owhadi, B. Hamzi, S. M. Hristova-Veleva, and J. Turk. Kernel flows to infer the structure of convective storms from satellite passive microwave observations. *AGU Fall Meeting Abstracts*, 2021.
- [18] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- [19] V. Vovk, A. Gammerman, and G. Shafer. *Algorithmic Learning in a Random World*. Springer, 2005.
- [20] H. Wendland. *Scattered Data Approximation*. Cambridge University Press, 2004.
- [21] D. H. Wolpert. Stacked generalization. *Neural Networks*, 5(2):241–259, 1992.
- [22] G. R. Yoo and H. Owhadi. Deep regularization and direct training of the inner layers of neural networks with kernel flows. *Physica D: Nonlinear Phenomena*, 426:132952, 2021.

A Proofs

A.1 Proof of Proposition 4.1

By convexity of $\ell(\cdot, v)$,

$$\ell(\widehat{G}_S(u), G(u)) \leq \frac{1}{2} \ell(\widehat{G}(u), G(u)) + \frac{1}{2} \ell(T\widehat{G}(Su), G(u)).$$

Since T is an involution, the equivariance $G(Su) = TG(u)$ applied at u gives $TG(Su) = T^2G(u) = G(u)$, hence

$$\ell(T\widehat{G}(Su), G(u)) = \ell(T\widehat{G}(Su), TG(Su)) = \ell(\widehat{G}(Su), G(Su)),$$

using the T -invariance of the loss. Taking expectations and using the S -invariance of μ (so that $Su \sim \mu$ when $u \sim \mu$),

$$\mathbb{E} \ell(\widehat{G}_S(u), G(u)) \leq \frac{1}{2} \mathbb{E} \ell(\widehat{G}(u), G(u)) + \frac{1}{2} \mathbb{E} \ell(\widehat{G}(Su), G(Su)) = \mathbb{E} \ell(\widehat{G}(u), G(u)). \quad \square$$

A.2 Proof of Proposition 4.2

Strict positive definiteness of the restricted kernel matrix makes I_C the (unique) interpolant of the pairs indexed by C , so $I_C(z_j) = v_j$ for $j \in C$ and the terms of (1) indexed by C vanish, which is the first claim. For the second, write

$$e_2(\theta; C) = \sum_{i \in B \setminus C} g(C, i), \quad g(C, i) := \|v_i - I_C(z_i)\|_2^2.$$

Conditionally on C , the sum has exactly $b/2$ terms, so $e_2(\theta; C) = \frac{b}{2} \mathbb{E}_{i \sim \text{Unif}(B \setminus C)}[g(C, i)]$, and taking the expectation over C proves the identity. The gradient claim is immediate: for fixed (B, C) the map $\theta \mapsto e_2(\theta; C)$ is differentiable wherever the Cholesky factorization in I_C is (the kernel matrix stays positive definite in a neighborhood), and under a local integrable Lipschitz bound the derivative passes under the expectation, so $\mathbb{E}_{B,C}[\nabla_\theta e_2] = \nabla_\theta \mathbb{E}_{B,C}[e_2]$. $\square$

A.3 Proof of Proposition 4.3

(i) Since $\sum_m w_m = 1$, $F_w(u) - G(u) = \sum_m w_m (f_m(u) - G(u))$, and the triangle inequality gives $\|F_w(u) - G(u)\|_2 \leq \sum_m w_m \|f_m(u) - G(u)\|_2$. Dividing by $\|G(u)\|_2$ yields the pointwise claim; taking expectations gives the risk inequality, and bounding each $\ell(f_m(u), G(u))$ by B gives $\ell(F_w(u), G(u)) \leq B$.

(ii) Write $\ell_u(w) = \ell(F_w(u), G(u))$. First, ℓ_u is Lipschitz on Δ for the ℓ^1 norm: for $w, w' \in \Delta$, the reverse triangle inequality and the argument of (i) give

$$|\ell_u(w) - \ell_u(w')| \leq \frac{\|\sum_m (w_m - w'_m)(f_m(u) - G(u))\|_2}{\|G(u)\|_2} \leq B \|w - w'\|_1.$$

Next, discretize the simplex. For $k \in \mathbb{N}$ let $\mathcal{G}_k = \{w \in \Delta : kw \in \mathbb{Z}^M\}$; its cardinality is the number of compositions of k into M nonnegative parts, $\binom{k+M-1}{M-1} \leq (k+1)^{M-1}$. Given $w \in \Delta$, set $w'_i = \lfloor kw_i \rfloor / k$ for $i < M$ and $w'_M = 1 - \sum_{i < M} w'_i$; then $w' \in \mathcal{G}_k$ (the last coordinate is a multiple of $1/k$ and is $\geq w_M \geq 0$ because the first $M-1$ coordinates only decreased), and

$$\|w - w'\|_1 = \sum_{i < M} (w_i - w'_i) + (w'_M - w_M) = 2 \sum_{i < M} (w_i - w'_i) \leq \frac{2(M-1)}{k}.$$

By (i) the per-sample loss lies in $[0, B]$, so for each fixed w' Hoeffding's inequality gives $\mathbb{P}(|\widehat{R}(w') - R(w')| > t) \leq 2e^{-2mt^2/B^2}$, and a union bound over $\mathcal{G}_k$ shows that with probability at least $1 - \delta$,

$$\sup_{w' \in \mathcal{G}_k} |\widehat{R}(w') - R(w')| \leq B \sqrt{\frac{\log(2|\mathcal{G}_k|/\delta)}{2m}}.$$

On this event, for every $w \in \Delta$, approximating by its grid point and using the Lipschitz property for both R and $\widehat{R}$,

$$|\widehat{R}(w) - R(w)| \leq B \sqrt{\frac{\log(2|\mathcal{G}_k|/\delta)}{2m}} + \frac{4B(M-1)}{k}.$$

If w^* minimizes R over Δ , the empirical minimizer satisfies $R(F_{\widehat{w}}) \leq \widehat{R}(\widehat{w}) + \sup_{\Delta} |\widehat{R} - R| \leq \widehat{R}(w^*) + \sup_{\Delta} |\widehat{R} - R| \leq R(F_{w^*}) + 2 \sup_{\Delta} |\widehat{R} - R|$. Choosing $k = m(M-1)$ and using $|\mathcal{G}_k| \leq$

$(m(M-1)+1)^{M-1}$ gives

$$R(F_{\hat{w}}) \leq \min_{w \in \Delta} R(F_w) + \frac{8B}{m} + B \sqrt{\frac{2((M-1)\log(m(M-1)+1) + \log(2/\delta))}{m}},$$

which is the claim. $\square$

A.4 Proof of Proposition 4.4

Since $\sum_m w_m = 1$, the stack's normalized residual is $\rho_{f_w}(u) = \sum_m w_m \rho_{f_m}(u)$, hence

$$\mathbb{E} \ell(f_w(u), G(u))^2 = \mathbb{E} \left\| \sum_m w_m \rho_{f_m}(u) \right\|_2^2 = \sum_{m,k} w_m w_k \mathbb{E} \langle \rho_{f_m}(u), \rho_{f_k}(u) \rangle = w^\top S w.$$

For (i), parameterize $w = (1-t, t)$; $q(t) = w^\top S w$ is a quadratic in t with $q'(0) = 2(\varrho e_1 e_2 - e_1^2)$, negative precisely when $\varrho < e_1/e_2$, and minimizing the quadratic over $t \in [0, 1]$ gives the stated value. For (ii), $S = e^2((1-\varrho)I + \varrho \mathbf{1}\mathbf{1}^\top)$, so $w^\top S w = e^2((1-\varrho)\|w\|_2^2 + \varrho)$ on the simplex, minimized by the uniform weights, where $\|w\|_2^2 = 1/M$. $\square$

A.5 Proof of Proposition 4.5

Both displays are the native-space estimate for kernel ridge regression: for a Matérn kernel of smoothness s on a bounded domain, with the nugget chosen as in the pipeline, an estimator $\hat{G}_n$ of a target F in the native space obeys $\mathbb{E}\|F - \hat{G}_n\| \leq C h_X^s \|F\|_{\mathcal{N}}$, with h_X the fill distance of the design in the kernel's metric [20], and for n samples from a density bounded above and below on a set of effective dimension d' , quasi-uniformity gives $h_X \asymp n^{-1/d'}$ with high probability.

For the isotropic kernel the metric is Euclidean on the full d -dimensional domain, so $h_X \asymp n^{-1/d}$; the target is $F = g(A\cdot)$, whose Matérn native norm is equivalent to its H^s norm, and each derivative of order up to s brings down at most a factor σ_1 , so $\|g(A\cdot)\|_{H^s} \asymp \sigma_1^s$. For the adapted kernel $k_A(u, u') = k(Au, Au')$ the estimator is the isotropic estimator of g on the design $\{Au_i\}$: in the rescaled coordinates the target has unit norm and its variation is confined to the r directions above the truncation level, so the fill distance that enters the estimate is the one in those r directions, $h_X \asymp n^{-1/r}$. Dividing the two bounds gives the ratio. $\square$

A.6 Proof of Theorem 4.6

Fix u and write $\kappa = k(X, u) \in \mathbb{R}^n$, $A = K + n\lambda I$, $w = A^{-1}\kappa$. For each component j , membership $r_j \in \mathcal{H}_k$ and the reproducing property give

$$r_j(u) - \hat{r}_{\lambda,j}(u) = \left\langle r_j, k(u, \cdot) - \sum_{i=1}^n w_i k(u_i, \cdot) \right\rangle_{\mathcal{H}_k},$$

because $\hat{r}_{\lambda,j}(u) = \kappa^\top A^{-1} r_j(X) = \sum_i w_i r_j(u_i)$ and $r_j(u_i) = \langle r_j, k(u_i, \cdot) \rangle$. Cauchy-Schwarz yields

$$|r_j(u) - \hat{r}_{\lambda,j}(u)| \leq \|r_j\|_{\mathcal{H}_k} \left\| k(u, \cdot) - \sum_i w_i k(u_i, \cdot) \right\|_{\mathcal{H}_k},$$

and the second factor is independent of j ; call it $\rho(u)$. Expanding,

$$\rho(u)^2 = k(u, u) - 2w^\top \kappa + w^\top K w = k(u, u) - \kappa^\top A^{-1}(2A - K)A^{-1}\kappa.$$

Since $2A - K = A + n\lambda I$,

$$\rho(u)^2 = k(u, u) - \kappa^\top A^{-1} \kappa - n\lambda \|A^{-1} \kappa\|_2^2 = \tilde{P}_\lambda(u)^2 \leq P_\lambda(u)^2.$$

Summing the squared componentwise bounds,

$$\|r(u) - \hat{r}_\lambda(u)\|_2^2 = \sum_j (r_j(u) - \hat{r}_{\lambda,j}(u))^2 \leq \rho(u)^2 \sum_j \|r_j\|_{\mathcal{H}_k}^2 = \tilde{P}_\lambda(u)^2 \|r\|_K^2. \quad \square$$

Note that the argument does not use interpolation: it holds for every $\lambda \geq 0$, with the (slightly sharper) factor $\tilde{P}_\lambda$ showing that smoothing can only tighten this particular bound relative to the posterior standard deviation P_λ that we report.

A.7 Proof of Proposition 4.7

Write $s_i = \|e(\tilde{u}_i)\|_2 / P_\lambda(\tilde{u}_i)$ for the validation scores and $s^* = \|e(u^*)\|_2 / P_\lambda(u^*)$ for the test score. Because $m, \hat{r}_\lambda, X$ and P_λ are fixed and the inputs $\tilde{u}_1, \dots, \tilde{u}_m, u^*$ are exchangeable and independent of them, the scores $s_1, \dots, s_m, s^*$ are exchangeable. The event $G(u^*) \in C(u^*)$ is exactly $\{s^* \leq q\}$, where $q = s_{(\lceil(1-\alpha)(m+1)\rceil)}$ is the $k := \lceil(1-\alpha)(m+1)\rceil$ -th order statistic of the validation scores.

Consider the augmented sample $s_1, \dots, s_m, s^*$ of size $m+1$ and let $\text{rank}(s^*)$ be its rank (ties broken uniformly at random, which only helps). By exchangeability the rank is uniform on $\{1, \dots, m+1\}$, so $\mathbb{P}(\text{rank}(s^*) \leq k) = k/(m+1)$. If s^* is among the k smallest of the $m+1$ values then at most $k-1$ of the s_i are below it, so $s^* \leq s_{(k)} = q$; conversely $s^* \leq q$ forces $\text{rank}(s^*) \leq k$ except possibly through ties. Hence

$$\mathbb{P}(s^* \leq q) \geq \mathbb{P}(\text{rank}(s^*) \leq k) = \frac{k}{m+1} = \frac{\lceil(1-\alpha)(m+1)\rceil}{m+1} \geq 1-\alpha,$$

which is the lower bound. When the scores are almost surely distinct there are no ties, $\{s^* \leq q\} = \{\text{rank}(s^*) \leq k\}$ exactly, and $k/(m+1) < ((1-\alpha)(m+1)+1)/(m+1) = 1-\alpha+1/(m+1)$, giving the upper bound. $\square$

A.8 Proof of Corollary 4.9

The nonzero eigenvalues of $K/n = XX^\top/n$ coincide with those of the sample covariance matrix $S = X^\top X/n \in \mathbb{R}^{p \times p}$, and zero eigenvalues contribute nothing to $d_{\text{eff}}(\lambda) = \sum_i \lambda_i(K)/(\lambda_i(K) + n\lambda)$, so

$$\frac{d_{\text{eff}}(\lambda)}{n} = \frac{p}{n} \cdot \frac{1}{p} \sum_{j=1}^p \frac{\lambda_j(S)}{\lambda_j(S) + \lambda} = \frac{p}{n} \left(1 - \lambda \hat{m}_S(-\lambda)\right),$$

by the computation of Lemma 4.8 applied to S , with $\hat{m}_S$ the Stieltjes transform of the empirical spectral distribution of S . By the Marčenko–Pastur theorem [13], this distribution converges weakly, almost surely, to the law with ratio γ and scale σ^2 , and since $x \mapsto (x + \lambda)^{-1}$ is bounded and continuous on $[0, \infty)$, $\hat{m}_S(-\lambda) \rightarrow m(-\lambda)$. It remains to evaluate $m(-\lambda)$. The Stieltjes transform of the Marčenko–Pastur law satisfies the self-consistent equation

$$m(z) = \frac{1}{\sigma^2(1 - \gamma - \gamma z m(z)) - z},$$

i.e. $\gamma\sigma^2 z m^2 + (z - \sigma^2(1 - \gamma))m + 1 = 0$. At $z = -\lambda$ the discriminant is $(\lambda + \sigma^2(1 - \gamma))^2 + 4\gamma\sigma^2\lambda > 0$, and of the two real roots

$$m(-\lambda) = \frac{\sigma^2(1 - \gamma) + \lambda \pm \sqrt{(\lambda + \sigma^2(1 - \gamma))^2 + 4\gamma\sigma^2\lambda}}{-2\gamma\sigma^2\lambda}$$

only the one with the minus sign in the numerator is positive, as $m(-\lambda) = \int (x + \lambda)^{-1} dF(x) > 0$ requires; rearranging gives the stated expression. $\square$

A.9 Proof of Lemma 4.8

Diagonalize K with eigenvalues λ_i . Then

$$\text{tr}(K(K + n\lambda I)^{-1}) = \sum_{i=1}^n \frac{\lambda_i}{\lambda_i + n\lambda} = \sum_{i=1}^n \left(1 - \frac{n\lambda}{\lambda_i + n\lambda}\right) = n - \lambda \sum_{i=1}^n \frac{1}{\lambda_i/n + \lambda} = n(1 - \lambda \widehat{m}(-\lambda)),$$

using $\widehat{m}(-\lambda) = \frac{1}{n} \sum_i (\lambda_i/n + \lambda)^{-1}$. The limit statement follows from the weak convergence of the empirical spectral distribution together with the boundedness and continuity of $x \mapsto (x + \lambda)^{-1}$ on $[0, \infty)$ for fixed $\lambda > 0$. $\square$

B Implementation details

All experiments ran on a single laptop (one NVIDIA RTX PRO 2000, 8 GB; 16 CPU cores, 64 GB RAM), in single precision for network training and double precision for all kernel solves and error computations. The dataset is the distributed **StructuralMechanics** file pair (40000 samples); inputs are reduced to $\mathbb{R}^{41}$ after verifying the broadcast structure exactly. Splits: the training block is samples 1–20000, the test set is samples 20001–40000. High-data protocol: a fixed permutation of the training block reserves 1000 samples for validation, 19000 for fitting. Low-data protocol: the first 1250 samples of the training block, of which the last 250 form the validation split. The test set is never touched during development; each configuration is evaluated on it once, after selection on validation.

FNO. Width 64, 14 modes per dimension, 4 spectral layers with pointwise linear skips and GELU, lift from 3 channels (broadcast load, two coordinates), projection $64 \rightarrow 128 \rightarrow 1$. AdamW, learning rate 1.5×10^{-3} (2×10^{-3} with batch 256), weight decay 10^{-6} , cosine schedule to 10^{-6} , 300 epochs, batch 256. 6.45M parameters.

Transformer (implemented, not in the reported ensemble). Encoder: 41 tokens (load value and 10 sine/cosine pairs of the coordinate), 5 pre-norm blocks, dimension 192, 4 heads, MLP ratio 4. Decoder: grid queries from Fourier features of both coordinates, two cross-attention blocks, pointwise head $192 \rightarrow 192 \rightarrow 1$; 3.16M parameters. The cross-attention over 1681 grid queries is the most compute-intensive of our means, and the hardware available for this study (a single laptop GPU that proved unstable under sustained load) did not let us train it to the level of the other members; we therefore exclude it from the reported ensemble and note it as the natural fourth architecture family for a follow-up on stable hardware.

UNet. Three convolutional scales ($41 \rightarrow 21 \rightarrow 11$ by average pooling, ceil mode) and a bottleneck at 6×6 ; two 3×3 convolutions with GroupNorm(8) and SiLU per block; widths 48/96/192/384; bilinear upsampling with skip concatenation; 1×1 output head. Input channels as for the FNO. AdamW, learning rate 1.5×10^{-3} , weight decay 10^{-5} , cosine schedule, 200 epochs, batch 256. 4.38M parameters.

MSE variant. The residual MLP trained with mean squared error on standardized targets in place of the metric loss, otherwise identical recipe; it enters the ensemble as a loss-diversity member and is also the strongest plain MLP we obtain.

MLP. Input layer $41 \rightarrow 1024$, three residual SiLU blocks of width 1024, output $1024 \rightarrow 1681$. AdamW, learning rate 10^{-3} , weight decay 10^{-5} , cosine schedule, 400 epochs, batch 256. 4.91M parameters. A wider variant (width 1536, five residual blocks, 14.5M parameters) is trained under the same recipe.

Refiner. The same residual trunk with input layer $(41 + 1681) \rightarrow 1024$: the load is concatenated with the kernel method’s stress prediction for that load. Training uses four-fold out-of-fold kernel predictions for the input channel and full-data kernel predictions at evaluation; otherwise identical to the MLP. 6.64M parameters. Reflection augmentation flips the load and permutes both the kernel channel and the target by the row-reversal index.

Common training details. Loss: mean over the batch of $\|\hat{v} - v\|_2 / \|v\|_2$ in original units (predictions denormalized inside the loss; targets standardized by the per-pixel training mean and global standard deviation), except for the MSE variant above. Reflection augmentation with probability 1/2; reflection averaging at evaluation. Model selection: validation error computed every 10 epochs, best checkpoint kept. A single seed is trained per architecture; the ensemble’s diversity comes from the architectures and losses rather than from reseeding, and the correlation analysis of Section 5.3 indicates same-architecture reseeds would be more correlated still.

Stacking. Global weights: convex, initialized at the simplex minimizer of the measured second-moment matrix S (Proposition 4.4) and polished by a short random search on the validation metric. Per-pixel weights: affine in the members at each grid point, ridge parameter 10^{-3} , fitted on half the validation split and accepted only if they beat the global weights on the held-out half, then refitted on the full split.

Kernel stages. Matérn-5/2 on standardized inputs. Scale grid $\{0.5, 1, 2, 4\} \times$ the median pairwise distance (estimated on 2000 points), nugget grid $\{10^{-7}, 10^{-5}, 10^{-3}\}$ (scaled by n), tuned on validation using an 8000-sample subsample of the training set, refit at the chosen pair on the full training set in double precision. The pure kernel baseline (Table 1) uses the grid $\{0.5, 0.75, 1, 1.5, 2\} \times$ median and nuggets $\{10^{-8}, 10^{-6}, 10^{-4}\}$ directly at $n = 19000$. Feature stage: penultimate activations of the ensemble members (FNO: spatially averaged pre-projection channels, 128; transformer: mean-pooled encoder tokens, 192; MLP: trunk output, 1024), concatenated, standardized, same kernel family and tuning.

Uncertainty. Posterior standard deviation (2) computed from the Cholesky factor of $K + n\lambda I$ by triangular solves. Conformal scaling: the 0.9-quantile of $\|e\|_2/P_\lambda$ on the validation split multiplies P_λ on test; coverage is the fraction of test samples whose absolute error norm falls below the scaled band.